```
  return data;
}

processGrid.size = function(newSize) {
        if (!arguments.length) return gridSize;

    gridSize = newSize;
    return this;
}

return processGrid;
}
```

Getter/setter function for layout size

You can see the updated grid layout in action by slightly changing our code for calling the layout, as shown in the following listing. We set the size, and when we create our circles, we use the x and y values directly instead of using scaled values.

Listing 10.6 Calling the new grid layout

```
var grid = d3.layout.grid();
grid.size([400,400]);
var griddedData = grid(data.tweets);

d3.select("svg")
   .append("g")
   .attr("transform", "translate(50,50)")
   .selectAll("circle").data(griddedData)
   .enter()
   .append("circle")
   .attr("cx", function(d) {return d.x})
   .attr("cy", function(d) {return d.y})
   .attr("r", 20)
   .style("fill", "pink");

var fakeTweets = [];
for (var x = 0;x<12;x++) {
    var tweet = {id: x, content: "Fake Tweet #" + x};
    fakeTweets.push(tweet);
}

var doubledArray = data.tweets.concat(fakeTweets);
var newGriddedData = grid(doubledArray);

d3.select("g").selectAll("circle").data(newGriddedData)
   .enter()
   .append("circle")
   .attr("cx", 0)
   .attr("cy", 0)
   .attr("r", 20)
   .style("fill", "darkred");

d3.select("g").selectAll("circle")
   .transition()
   .duration(1000)
   .attr("cx", function(d) {return d.x})
   .attr("cy", function(d) {return d.y})
   .each("end", resizeGrid1);
```

Sets layout size

Position circles with their x/y values

At the end of the transition, calls resizeGrid1